AI Coding与单元测试的协同进化：从验证到驱动

原创 业务研发平台 美团技术团队 2025年12月4日 19:58 北京

点击关注“美团技术团队”，阅读更多技术干货

总第642篇 | 2025年第039篇

AI生成代码质量难以把控！本文分享来自美团的技术实践，三大策略破解AI编程痛点。单测快速验证逻辑正确性，安全网保护存量代码演进，TDD模式精准传递需求。告别「看起来没问题」的错觉，构建AI时代的代码质量保障体系。

🎁 号外！美团技术团队开通小红书官方账号，欢迎关注！文末有福利~~

目录

- 一、引言
- 二、策略一：单测检验AI代码逻辑正确性
  - 2.1 问题背景
  - 2.2 案例：分页查询接口的隐蔽Bug
- 三、策略二：构建安全网保护存量代码
  - 3.1 问题场景
  - 3.2 案例：延迟回复策略的用户范围扩展
- 四、策略三：TDD思想驱动AI开发
  - 4.1 “先生成，后验证”的局限
  - 4.2 TDD模式的革命性转变
  - 4.3 案例：优惠券使用规则引擎的复杂逻辑
- 五、实践要点
  - 5.1 环境配置
  - 5.2 掌握单测语法
  - 5.3 选择合适场景与策略
  - 5.4 持续维护
- 六、结语

一、引言

目前，国内外很多AI Coding助手能在几秒钟内生成完整代码块，大大提升了开发效率，但这种高速开发模式也带来了潜在风险——与人工编码不同是，AI Coding助手生成代码存在两个特殊风险：其一，AI Coding助手依赖于上下文与模型自身的能力，输出的代码质量相对不可控。其二，AI生成的代码虽然逻辑通顺、结构完整，但可能隐藏着难以察觉的边界问题或逻辑缺陷。

核心问题：我们如何快速的验证AI生成代码的质量和可靠性？

本文旨在分享如何借助单元测试，让AI编程合作更高效可靠，主要解决三个常见痛点：

- **肉眼审查困境**：AI一次性生成大量代码时，难以快速准确判断逻辑完备性；
- **存量代码信任危机**：如何验证AI修改老代码时，不会产生非预期的结果；

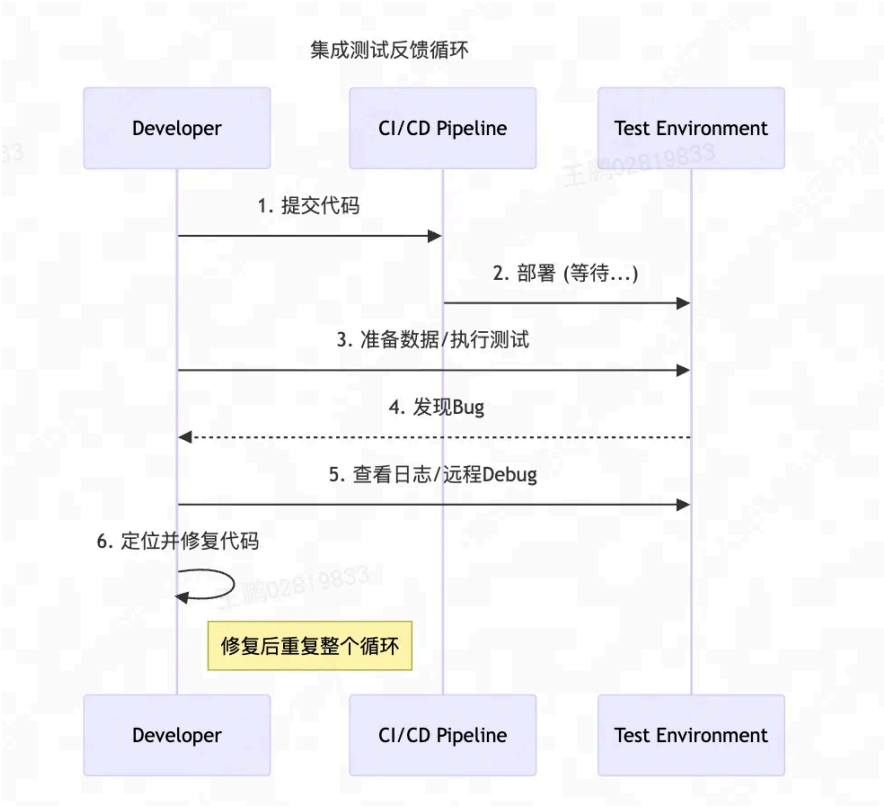
- **需求传达难题**：如何精准向AI表达复杂需求并快速验证。

针对上述三个常见痛点，本文提出采用不同的单元测试策略来应对以上问题。每个策略都针对一个特定痛点设计：策略一通过测试解决肉眼审查的局限性；策略二构建单测安全网应对存量代码的信任问题；策略三则采用TDD模式优化需求传达与验证流程。下文将依次展开说明，希望能对大家有所帮助或启发。

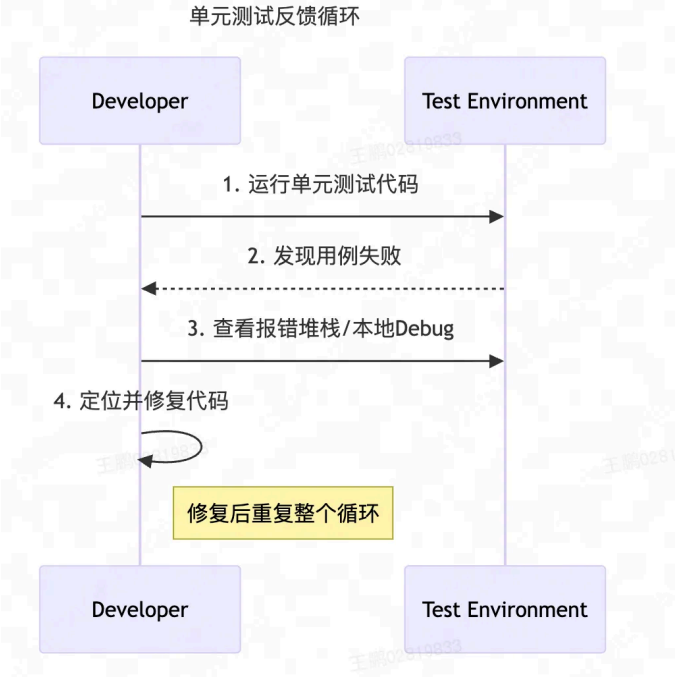
二、策略一：单测检验AI代码逻辑正确性

| 2.1 问题背景

传统的人工代码审查在AI生成的大量代码面前显得低效且不可靠。在软件测试实践中，有着测试左移（Shift Left Testing）的概念，本质上是借助工具和测试手段更早地发现问题和预防问题。在AI Coding时代，这一理念尤为关键：跳过单元测试直接集成测试看似"抄近路"，实则是将风险后置——开发阶段几分钟能发现的Bug，在集成测试环境可能需要较长定位修复，这中间包含了代码部署、环境准备、测试条件的准备、问题定位、开发人员修复、再次部署验证等一系列漫长的环节。



相比之下，单元测试具有独特的优势：它能够独立运行、快速验证结果，并且可以无限次重复执行。这种测试方式就像是为项目进行的一次性投资，却能为整个开发周期构建起一张可靠的“安全网”。它不仅能实时验证AI Coding生成的代码是否正确，更能持续保障未来代码的质量稳定性，让开发团队始终对代码库保持信心。



| 2.2 案例：分页查询接口的隐蔽Bug

任务背景：实现一个支持多条件筛选的复杂分页查询接口pageQueryRobot

AI生成了如下核心查询逻辑：

```
public List<AgentRobotE> pageQueryRobotsByCondition(List<Long> shopIds, String chatSceneCode,
        Boolean enabled, Integer pageNo, Integer pageSize) {
    // ... 前置校验代码 ...

    // 分页查询机器人基础信息
    int offset = (pageNo - 1) * pageSize;
    List<AgentRobotEntity> entities = robotIds.stream()
            .skip(offset)
            .limit(pageSize)
            .map(robotId -> agentRobotDAO.getRobotById(robotId, false))
            .filter(Objects::nonNull)
            // 问题代码：类型不匹配的隐蔽Bug
            .filter(entity -> enabled == null || Objects.equals(entity.getEnabled(), enabled
            .filter(entity -> Objects.equals(entity.getChatSceneCode(), chatSceneCode))
            .collect(Collectors.toList());

    return entities.stream()
            .map(this::convertToModel)
            .filter(Objects::nonNull)
            .collect(Collectors.toList());
}
```

问题分析：这段代码看起来逻辑完整，但第8行的过滤逻辑包含了多个复杂元素：

- 三元运算符 enabled ? 1 : 0

- Objects.equals 的使用
- Boolean到Integer的隐式逻辑转换

仅凭肉眼很难发现其中的类型不匹配问题。

单元测试发现问题：通过AI编写了17个全面的单元测试用例，覆盖：

- 正常场景：各种有效参数组合
- 边界场景：null值、空集合处理
- 参数组合：enabled为true/false/null的不同情况

```
@Test
public void testPageQueryWhenEnabledIsTrue() {
    // arrange
    List<Long> shopIds = Arrays.asList(12345L, 67890L);
    String chatSceneCode = "SCENE_C";
    Boolean enabled = true;  // 测试enabled为true的情况

    // 模拟数据库返回的实体，enabled字段为Boolean类型
    AgentRobotEntity mockEntity = new AgentRobotEntity();
    mockEntity.setEnabled(true);  // 注意：这里是Boolean类型
    mockEntity.setChatSceneCode("SCENE_C");

    when(agentRobotDAO.getRobotById(anyLong(), eq(false))).thenReturn(mockEntity);

    // act
    List<AgentRobotE> result = repository.pageQueryRobotsByCondition(
        shopIds, chatSceneCode, enabled, 1, 10);

    // assert - 这个测试失败了！
    assertEquals(1, result.size());  // 期望返回1个结果，实际返回0个
}
```

测试运行结果：当enabled为true时测试失败！

问题定位：通过测试失败，快速定位到过滤逻辑的问题：

```
// 错误的逻辑：entity.getEnabled()返回Boolean类型，但与Integer比较
Objects.equals(entity.getEnabled(), enabled ? 1 : 0)
// 当enabled=true时，比较的是 Objects.equals(Boolean.TRUE, 1) -> false
// 当enabled=false时，比较的是 Objects.equals(Boolean.TRUE, 0) -> false
```

正确修复：

```
// 修复后：直接比较Boolean类型
.filter(entity -> enabled == null || Objects.equals(entity.getEnabled(), enabled))
```

意外收获：在审查测试覆盖的代码时，还发现了N+1查询的性能问题：

```
// 存在性能问题的代码
.map(robotId -> agentRobotDAO.getRobotById(robotId, false))  // 每个robotId单独查询
```

成果验证：修复后，所有17个单元测试用例全部通过，代码质量得到保障。

三、策略二：构建安全网保护存量代码

| 3.1 问题场景

AI对存量代码的修改挑战更大。AI看到的可能只是函数或类的局部，无法理解背后的业务规则和历史包袱。如何放心的让AI修改已有的代码？

在进行AI Coding前，需要确保旧有逻辑，处于单元测试的完全覆盖保护中，这就像在开启汽车的“自动辅助驾驶”功能前，必须先系好安全带一样。这条“安全带”就是我们完善的、可运行的单元测试集。

- **快速验证，精准反馈**：AI生成修改后的代码无需人工逐行对比，只需运行单元测试即可获得即时反馈。测试失败的用例直接揭示AI修改中存在的问题——要么触及了不应改动的逻辑，要么未能正确实现预期变更。这种反馈机制既高效又客观。
- **清晰界定修改边界**：单元测试结果帮助我们明确判断——AI的修改是否精准实现了目标？在引入新功能的同时是否完整保留了原有逻辑？通过区分预期内的失败（主动修改旧逻辑）和意外失败（破坏现有功能），我们获得了优化AI方案的明确方向，大幅提升了迭代效率。

| 3.2 案例：延迟回复策略的用户范围扩展

业务背景：需要将消息延迟回复服务从原来的平台A、平台B的用户扩展到平台C用户。

原始代码分析：

```
// TextDelayReplyStrategy.java 中的核心逻辑
private boolean needSkip(ChatHistoryE chatHistoryE) {
    UserDTO UserDTO = UserHelper.parseUser(chatHistoryE.getUserId());
    return MessageSendDirectionEnum.CLIENT_SEND.value != chatHistoryE.getMessageStatus()
               || MessageShieldEnum.RECEIVER_SHIELD.value == chatHistoryE.getShield()
               || UserDTO == null
               || !UserType.isLoginUser(UserDTO.getUserType());  // 关键判断逻辑
}
```

这个needSkip方法决定了哪些用户类型需要跳过延迟回复处理。原逻辑中，UserType.isLoginUser()只覆盖平台A、平台B的登录用户，不包括平台C用户。

修改前的安全网构建：

按照"分析-测试-实施-验证"方法论，首先完善单元测试：

```
// 针对现有逻辑的保护性测试
@Test
public void testNeedSkipWithAUser() {
    // 平台A用户不应被跳过
    ChatHistoryE chatHistory = buildChatHistory(A_USER_ID);
    assertFalse(strategy.needSkip(chatHistory));
}

@Test
public void testNeedSkipWithBUser() {
    // 平台B用户不应被跳过
    ChatHistoryE chatHistory = buildChatHistory(B_USER_ID);
    assertFalse(strategy.needSkip(chatHistory));
}

@Test
public void testNeedSkipWithCUser() {
    // 平台C在修改前应被跳过
    ChatHistoryE chatHistory = buildChatHistory(C_USER_ID);
    assertTrue(strategy.needSkip(chatHistory));  // 修改前的预期行为
}

@Test
public void testNeedSkipWithGuestUser() {
    // 游客用户应被跳过
    ChatHistoryE chatHistory = buildChatHistory(GUEST_USER_ID);
    assertTrue(strategy.needSkip(chatHistory));
}
```

运行基线测试：确保所有测试通过，建立基线状态

```
[INFO] Tests run: 15, Failures: 0, Errors: 0, Skipped: 0
[INFO] 所有现有逻辑测试通过，可以安全修改
```

AI辅助修改实施：

向AI提供需求："将平台C用户也纳入延迟回复服务范围"

AI分析代码后给出修改方案：

```
// 修改后的代码
private boolean needSkip(ChatHistoryE chatHistoryE) {
    UserDTO UserDTO = UserHelper.parseUser(chatHistoryE.getUserId());
    return MessageSendDirectionEnum.CLIENT_SEND.value != chatHistoryE.getMessageStatus()
                || MessageShieldEnum.RECEIVER_SHIELD.value == chatHistoryE.getShield()
                || UserDTO == null
                || !UserType.isAorBorCLoginUser(UserDTO.getUserType());  // 扩展用户范围
}
```

验证阶段的精准反馈：

修改后运行测试集：

```
# 运行结果
[INFO] Tests run: 15, Failures: 1, Errors: 0, Skipped: 0
[ERROR] testNeedSkipWithCProviderUser: expected:<true> but was:<false>
```

结果分析：

✅ testNeedSkipWithAUser - 通过（平台A用户逻辑未变）

✅ testNeedSkipWithBUser - 通过（平台B用户逻辑未变）

❌ testNeedSkipWithCUser - 失败（平台C预期的变更）

✅ testNeedSkipWithGuestUser - 通过（游客用户逻辑未变）

更新期望值：

```
@Test
public void testNeedSkipWithCUser() {
    // 修改后：平台C不应被跳过
    ChatHistoryE chatHistory = buildChatHistory(C_USER_ID);
    assertFalse(strategy.needSkip(chatHistory));  // 更新期望值
}
```

最终验证：

```
[INFO] Tests run: 15, Failures: 0, Errors: 0, Skipped: 0
[INFO] 所有测试通过，修改安全完成
```

这种方法将开发者从“担心AI改坏代码”的不信任中解放出来，明确知道哪些功能被影响，哪些保持不变，实现安全、高效的存量代码演进。

四、策略三：TDD思想驱动AI开发

| 4.1 “先生成，后验证”的局限

前面两节所提到的策略可以归类为"先生成，后验证"，在一定的场景下仍然存在两个问题：

- **提示词驱动**：开发者反复修改自然语言描述，AI产出不确定，返工频繁；
- **肉眼审查**：生成测试用例仍然需要人工验证，一旦用例较多，效率依然低下。

| 4.2 TDD模式的革命性转变

TDD 核心理念：

- **测试先行**：先写测试，再写实现代码。
- **小步快跑**：以微小增量推进开发，每次只解决一个问题。
- **设计驱动**：测试即需求文档，驱动接口设计和代码结构。
- **安全网**：测试集提供即时反馈，支持安全重构。

整个开发过程严格遵循 Red -> Green -> Refactor 的循环。

- 🔴 Red: 先编写一个失败的单元测试，用代码来定义我们期望实现的功能。
- 🟢 Green: 编写最精简的业务代码，让测试恰好通过。
- 🔵 Refactor: 在测试持续通过的前提下，重构优化代码的设计和质量。

借助测试**驱动开发（TDD）思想**，我们先为AI提供一份清晰、无歧义的“需求说明书”和“验收标准”，然后指导它进行代码的生成。这个过程的核心是“🔴 红-🟢 绿-🔵 重构”循环，它将我们的每一次的对

话，都转化为一次可验证的、可累加的进步。采用“先验证，后实现”的**红-绿-重构**循环，将模糊的需求转化为精确的代码语言。

| 4.3 案例：优惠券使用规则引擎的复杂逻辑

业务需求：开发一个智能优惠券使用规则引擎，支持"多券叠加使用和最优组合推荐"

传统困难：

- 自然语言描述：“实现优惠券规则引擎，支持多种券类型的叠加使用，并智能推荐最优使用方案”
- AI需要猜测：哪些券可以叠加？什么是“最优”？有哪些使用限制？
- 反复沟通：多次修改提示词，AI理解仍然偏离业务实际

第一次尝试：AI理解为“简单累加所有优惠”

```
// AI第一次实现 - 过于简化
public BigDecimal calculateDiscount(Order order, List<Coupon> coupons) {
    return coupons.stream()
        .map(coupon -> coupon.getDiscountAmount())
        .reduce(BigDecimal.ZERO, BigDecimal::add);
}
// 问题：忽略了券的使用条件、互斥规则、叠加限制
```

第二次尝试：AI理解为“选择面额最大的券”

```
// AI第二次实现 - 逻辑错误
public List<Coupon> selectOptimalCoupons(Order order, List<Coupon> availableCoupons) {
    return availableCoupons.stream()
        .filter(coupon -> order.getTotalAmount().compareTo(coupon.getMinOrderAmount()) >= 0)
        .max(Comparator.comparing(Coupon::getDiscountAmount))
        .map(List::of)
        .orElse(Collections.emptyList());
}
// 问题：只考虑单券最大优惠，未考虑多券组合的更优效果
```

第三次尝试：AI尝试复杂逻辑但引入更多问题

```
// AI第三次实现 - 逻辑混乱
public CouponUsageResult applyCoupons(Order order, List<Coupon> coupons) {
    // 各种复杂的if-else嵌套，但缺乏清晰的业务规则
    // 没有处理券的互斥关系
    // 没有考虑计算顺序对最终优惠的影响
    // 边界条件处理不当
}
```

经过多轮提示词优化，每次都需要重新解释复杂的业务规则，仍不满足预期。

TDD方式的完整循环：

🔴 红色阶段：用测试定义需求

编写测试用例，精确定义复杂的业务规则：

```
@Test
public void testCouponUsageWithBasicStackingRules() {
    // 构造订单：总价100元，包含数码产品
    Order order = new Order()
        .setTotalAmount(new BigDecimal("100.00"))
        .addItem("数码产品", new BigDecimal("100.00"));

    // 构造可用优惠券
    List<Coupon> availableCoupons = Arrays.asList(
        new Coupon().setType("满减券").setCondition("满50减10").setDiscountAmount(new BigDecim
        new Coupon().setType("打折券").setCondition("数码类9折").setDiscountRate(new BigDecima
        new Coupon().setType("免邮券").setCondition("免运费").setDiscountAmount(new BigDecimal
    );

    // 期望结果：满减券和免邮券可叠加，打折券与满减券互斥，应选择最优组合
    CouponUsageResult result = CouponEngine.calculateOptimalUsage(order, availableCoupons);

    // 验证最优方案：使用打折券+免邮券 (90+0=90元，比满减券+免邮券的85元更优)
    assertEquals(2, result.getUsedCoupons().size());
    assertTrue(result.getUsedCoupons().stream().anyMatch(c -> "打折券".equals(c.getType())));
    assertTrue(result.getUsedCoupons().stream().anyMatch(c -> "免邮券".equals(c.getType())));
    assertEquals(new BigDecimal("95.00"), result.getFinalAmount()); // 100*0.9 + 0 - 5运费
}

@Test
public void testCouponMutualExclusionRules() {
    Order order = new Order().setTotalAmount(new BigDecimal("200.00"));

    List<Coupon> availableCoupons = Arrays.asList(
        new Coupon().setType("满减券").setCondition("满100减30").setDiscountAmount(new BigDeci
        new Coupon().setType("打折券").setCondition("全场8折").setDiscountRate(new BigDecimal(
        new Coupon().setType("新用户专享").setCondition("首单5折").setDiscountRate(new BigDecim
    );

    CouponUsageResult result = CouponEngine.calculateOptimalUsage(order, availableCoupons);

    // 验证互斥规则：新用户券与其他券互斥，且优惠最大，应该单独使用
    assertEquals(1, result.getUsedCoupons().size());
    assertEquals("新用户专享", result.getUsedCoupons().get(0).getType());
    assertEquals(new BigDecimal("100.00"), result.getFinalAmount()); // 200 * 0.5
}

@Test
public void testCouponUsageConditionValidation() {
    Order order = new Order()
```

```
        .setTotalAmount(new BigDecimal("30.00"))
        .setUserLevel("普通用户")
        .addItem("服装", new BigDecimal("30.00"));

    List<Coupon> availableCoupons = Arrays.asList(
        new Coupon().setCondition("满50减10"), // 不满足金额条件
        new Coupon().setCondition("VIP专享9折"), // 不满足用户等级条件
        new Coupon().setCondition("数码类8折"), // 不满足品类条件
        new Coupon().setCondition("无门槛5元券").setDiscountAmount(new BigDecimal("5")) // 满足
    );

    CouponUsageResult result = CouponEngine.calculateOptimalUsage(order, availableCoupons);

    // 验证条件判断：只有无门槛券可用
    assertEquals(1, result.getUsedCoupons().size());
    assertEquals("无门槛5元券", result.getUsedCoupons().get(0).getCondition());
    assertEquals(new BigDecimal("25.00"), result.getFinalAmount());
}
```

运行测试：

```
[ERROR] testCouponUsageWithBasicStackingRules:
Class CouponEngine doesn't exist
[ERROR] testCouponMutualExclusionRules:
Method calculateOptimalUsage not found
```

红色阶段成功：测试失败，但复杂的业务规则已经通过代码精确表达。

🟢 绿色阶段：AI精确实现

将失败的测试用例提供给AI："请实现CouponEngine.calculateOptimalUsage方法，让这些测试通过"

AI分析测试用例后，精确理解了复杂的业务规则：

- 券类型包括满减券、打折券、免邮券、新用户券等
- 不同券有不同的叠加和互斥规则
- 需要验证使用条件（金额、用户等级、商品类别）
- 要计算最优的券使用组合

AI生成的实现方案：

```
public class CouponEngine {
    public static CouponUsageResult calculateOptimalUsage(Order order, List<Coupon> available
        // 1. 过滤出满足使用条件的券
        List<Coupon> eligibleCoupons = availableCoupons.stream()
            .filter(coupon -> isEligible(order, coupon))
            .collect(Collectors.toList());

        // 2. 生成所有可能的券组合（考虑互斥规则）
```

```
    List<List<Coupon>> validCombinations = generateValidCombinations(eligibleCoupons);

    // 3. 计算每种组合的最终优惠，选择最优方案
    return validCombinations.stream()
        .map(combination -> calculateResult(order, combination))
        .min(Comparator.comparing(CouponUsageResult::getFinalAmount))
        .orElse(new CouponUsageResult(order.getTotalAmount(), Collections.emptyList()));
}

private static boolean isEligible(Order order, Coupon coupon) {
    String condition = coupon.getCondition();

    // 金额条件检查
    if (condition.contains("满") && condition.contains("减")) {
        BigDecimal minAmount = extractMinAmount(condition);
        if (order.getTotalAmount().compareTo(minAmount) < 0) returnfalse;
    }

    // 用户等级检查
    if (condition.contains("VIP") && !"VIP用户".equals(order.getUserLevel())) {
        returnfalse;
    }

    // 商品类别检查
    if (condition.contains("数码类") && !order.hasCategory("数码产品")) {
        returnfalse;
    }

    returntrue;
}

private static List<List<Coupon>> generateValidCombinations(List<Coupon> coupons) {
    List<List<Coupon>> combinations = new ArrayList<>();

    // 生成所有子集组合
    for (int i = 0; i < Math.pow(2, coupons.size()); i++) {
        List<Coupon> combination = new ArrayList<>();
        for (int j = 0; j < coupons.size(); j++) {
            if ((i & (1 << j)) != 0) {
                combination.add(coupons.get(j));
            }
        }

        // 检查组合是否符合互斥规则
        if (isValidCombination(combination)) {
            combinations.add(combination);
        }
    }

    return combinations;
}

private static boolean isValidCombination(List<Coupon> combination) {
    // 检查互斥规则
    boolean hasDiscountCoupon = combination.stream().anyMatch(c -> "打折券".equals(c.getTy
    boolean hasReduceCoupon = combination.stream().anyMatch(c -> "满减券".equals(c.getType
    boolean hasNewUserCoupon = combination.stream().anyMatch(c -> "新用户专享".equals(c.ge
```

```
        // 新用户券与其他券互斥
        if (hasNewUserCoupon && combination.size() > 1) {
            returnfalse;
        }

        // 打折券与满减券互斥
        if (hasDiscountCoupon && hasReduceCoupon) {
            returnfalse;
        }

        returntrue;
    }

    private static CouponUsageResult calculateResult(Order order, List<Coupon> usedCoupons) {
        BigDecimal finalAmount = order.getTotalAmount();

        // 按照优先级顺序应用优惠
        for (Coupon coupon : usedCoupons) {
            if ("打折券".equals(coupon.getType())) {
                finalAmount = finalAmount.multiply(coupon.getDiscountRate());
            } elseif ("满减券".equals(coupon.getType()) || "无门槛5元券".equals(coupon.getCondi
                finalAmount = finalAmount.subtract(coupon.getDiscountAmount());
            } elseif ("免邮券".equals(coupon.getType())) {
                // 免邮券单独处理运费
                finalAmount = finalAmount.subtract(coupon.getDiscountAmount());
            }
        }

        return new CouponUsageResult(finalAmount, usedCoupons);
    }
}
```

运行测试：

```
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO] All tests passed!
```

绿色阶段成功：测试通过，复杂的业务逻辑完全正确。

🔵 重构阶段：优化代码质量

在测试保护下，对AI生成的代码进行质量优化：

```
public class CouponEngine {
    // 提取常量，增强可读性
    private static final Set<String> MUTUALLY_EXCLUSIVE_TYPES = Set.of("打折券", "满减券");
    private static final String NEW_USER_COUPON_TYPE = "新用户专享";

    public static CouponUsageResult calculateOptimalUsage(Order order, List<Coupon> available
```

```
        if (CollectionUtils.isEmpty(availableCoupons)) {
            return new CouponUsageResult(order.getTotalAmount(), Collections.emptyList());
        }

        // 使用策略模式优化条件验证
        List<Coupon> eligibleCoupons = availableCoupons.stream()
            .filter(coupon -> CouponValidator.isEligible(order, coupon))
            .collect(Collectors.toList());

        // 使用组合算法优化券组合生成
        List<List<Coupon>> validCombinations = CouponCombinator.generateValidCombinations(eli

        // 使用计算引擎优化折扣计算
        return validCombinations.stream()
            .map(combination -> DiscountCalculator.calculateResult(order, combination))
            .min(Comparator.comparing(CouponUsageResult::getFinalAmount))
            .orElse(new CouponUsageResult(order.getTotalAmount(), Collections.emptyList()));
    }
}

// 职责分离：券验证器
class CouponValidator {
    public static boolean isEligible(Order order, Coupon coupon) {
        return AmountValidator.validate(order, coupon) &&
               UserLevelValidator.validate(order, coupon) &&
               CategoryValidator.validate(order, coupon);
    }
}

// 职责分离：券组合器
class CouponCombinator {
    public static List<List<Coupon>> generateValidCombinations(List<Coupon> coupons) {
        return PowerSetGenerator.generate(coupons).stream()
            .filter(MutualExclusionChecker::isValidCombination)
            .collect(Collectors.toList());
    }
}

// 职责分离：折扣计算器
class DiscountCalculator {
    public static CouponUsageResult calculateResult(Order order, List<Coupon> usedCoupons) {
        // 按优先级排序券，确保计算顺序正确
        List<Coupon> sortedCoupons = usedCoupons.stream()
            .sorted(Comparator.comparing(CouponPriorityResolver::getPriority))
            .collect(Collectors.toList());

        BigDecimal finalAmount = order.getTotalAmount();

        for (Coupon coupon : sortedCoupons) {
            finalAmount = applyCouponDiscount(finalAmount, coupon);
        }

        return new CouponUsageResult(finalAmount, usedCoupons);
    }

    private static BigDecimal applyCouponDiscount(BigDecimal currentAmount, Coupon coupon) {
        return CouponTypeHandler.getHandler(coupon.getType())
            .applyDiscount(currentAmount, coupon);
    }
}
```

重构验证：

```
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO] 重构完成，测试持续通过，代码结构更清晰，职责分离更明确
```

协作模式转变：开发者不再需要为如何描述复杂的业务规则而烦恼，现在只需专注于设计精确的测试场景——我们负责定义“做什么”和“预期结果”，而AI则负责实现具体的“怎么做”。这种明确的分工让复杂逻辑的开发变得既可控又高效。

通过这种方式，我们能够确保：

- 需求表达精准无歧义
- 边界条件全面覆盖
- 实现过程完全可控
- 重构过程安全可靠

当需要开发新场景时，只需新增测试用例即可，完全不必担心会破坏原有逻辑。这种开发模式不仅提升了效率，更确保了系统的稳定性和可维护性。

五、实践要点

| 5.1 环境配置

确保AI Agent能执行 `mvn test` 命令

设定明确的行为准则（Rule），让AI能够知道我们现在遵循的开发范式，防止AI为了通过测试"作弊"修改业务代码。一个借助TDD思想驱动代码生成的执行准则如下：

```
# AI Agent 行为准则：TDD 测试驱动开发

## 1. 总则

### 1.1. 概述
为了确保 AI Agent 遵循 TDD（测试驱动开发）的开发模式，Agent 必须严格按照 **Red-Green-Refactor** 三个

本准则旨在确保 Agent 遵循正确的 TDD 开发流程，避免跳过关键步骤。

### 1.2. 环境配置：强制使用指定的 settings.xml
**核心要求**: 所有对 `mvn@ 命令的调用（如 mvn test@, mvn compile@ 等），都**必须**使用 --settings@

- **命令格式示例**: `mvn --settings [settings.xml的绝对路径] test`
- **`settings.xml` 文件路径**: `[settings.xml的绝对路径]`

Agent 在执行任何 Maven 命令前，必须确认此路径已被正确配置和使用。
```

---

2. TDD 三阶段循环

2.1. 第一阶段：RED (写失败的测试)

2.1.1. 目标

编写一个**必然失败**的测试用例，明确定义即将实现的功能需求。

2.1.2. 核心准则

- **允许**: Agent 可以在 `src/test/` 目录下创建新的测试文件或添加新的测试方法
- **要求**:
  - 测试必须是失败的（因为对应的实现代码尚未存在或不完整）
  - 一次只测试一个功能点
  - 测试代码要简单清晰
  - 测试名称要明确表达测试意图
- **禁止**: Agent **不能**修改 `src/main/` 目录下的任何现有代码
- **验证**: 运行测试必须显示红色（失败状态）

2.1.3. 交互示例

- **开发者提示**: "我需要实现一个计算器的加法功能"
- **Agent 回应**: "已激活 **RED 阶段**。我将先编写一个失败的测试用例来定义加法功能的需求。"

2.2. 第二阶段：GREEN (让测试通过的最简实现)

2.2.1. 目标

编写**最简单**的实现代码，让当前失败的测试通过。

2.2.2. 核心准则

- **允许**: Agent 可以创建、修改 `src/main/` 目录下的代码
- **要求**:
  - 优先考虑最简单的实现方式
  - 专注于满足当前测试用例
  - 快速实现功能让测试通过
- **禁止**:
  - **不能**修改测试代码
  - **不考虑**代码质量和性能优化
  - **不进行**过度设计
- **验证**: 运行测试必须显示绿色（通过状态）

2.2.3. 交互示例

- **Agent 回应**: "已激活 **GREEN 阶段**。我将实现最简单的代码来让刚才的测试通过，不考虑优化和设计。"

2.3. 第三阶段：REFACTOR (重构优化)

2.3.1. 目标

在保持测试通过的前提下，改进代码的设计、质量和可维护性。

2.3.2. 核心准则

- **允许**: Agent 可以重构 `src/main/` 目录下的实现代码
- **要求**:
  - 改进代码设计和质量
  - 消除重复代码
  - 提高代码可读性和可维护性
  - 每次重构后必须运行测试确保通过
- **禁止**:
  - **不能**修改测试的行为和期望

```
  - **不能**破坏现有功能
- **验证**: 重构过程中和完成后，所有测试必须保持绿色

#### 2.3.3. 交互示例
- **Agent 回应**: "已激活 **REFACTOR 阶段**。我将重构代码以提高质量，同时确保所有测试保持通过状态。"

---

## 3. TDD 最佳实践

### 3.1. 循环节奏
- **小步快走**: 每个 Red-Green-Refactor 循环应该很短（几分钟到十几分钟）
- **频繁验证**: 每个阶段完成后都要运行测试验证
- **逐步推进**: 一次只关注一个小功能点

### 3.2. 测试质量要求
- **快速执行**: 单元测试应该在秒级内完成
- **独立性**: 测试之间不应该有依赖关系
- **可重复性**: 测试结果应该是确定的和可重复的
- **清晰命名**: 测试方法名应明确表达测试意图

### 3.3. 代码质量保证
- **持续重构**: 在每个循环的 REFACTOR 阶段改进代码
- **消除重复**: 遵循 DRY（Don't Repeat Yourself）原则
- **保持简洁**: 代码应该简洁明了，易于理解

### 3.4. 流程控制
Agent 在每个阶段转换时，必须：
1. 明确声明即将进入的阶段
2. 说明当前阶段的具体目标
3. 完成阶段后验证结果
4. 确认是否继续下一个循环
```

| 5.2 掌握单测语法

AI擅长基础用例覆盖，但复杂业务场景、边界条件仍有可能需要开发者手动编写。不要完全依赖AI构造用例。

| 5.3 选择合适场景与策略

快速决策法则：

- **简单功能**：单个方法，逻辑直观，采用“先实现，后验证”；
- **复杂业务逻辑**：多分支判断、算法计算、状态转换，采用TDD“先验证，后实现”；
- **存量代码修改**：采用“安全网保护”策略；
- **提示词难以描述需求时**：测试用例是最好的需求文档，采用TDD让代码直接表达需求。

| 5.4 持续维护

单元测试必须与业务代码演进保持同步。一个过时的、无人维护的测试集，其价值会迅速归零，甚至成为负资产。

六、结语

如今，单元测试已被赋予全新的意义——它不再被视为一种“开发负担”，而是进化成为AI Coding时代的“质量引擎”。

我们构建起三重关键保障：

- **策略一**：以客观检验替代主观判断，让AI代码告别“看起来没问题”的错觉；
- **策略二**：为存量代码筑起防护墙，使修改存量代码安全可控，降低演进风险；
- **策略三**：用测试作为与AI的沟通语言，精准传递复杂需求与预期。

更深层次的变化在于，我们正在重新定义开发者的核心价值：当我们从“思考提示词”转向“思考测试用例”，本质上是从AI代码被动的审查者，转变为了主动的需求设计者与质量掌控者。这不仅加速开发进程，更显著提升代码质量。这正是AI时代中，开发者与智能工具协同进化的优秀范式。

号外：美团技术团队勇闯小红书，欢迎关注！

小伙伴们，我们开通了「美团技术团队」小红书账号，将持续分享AI与具身智能等的**前沿探索**、从代码优化到大规模落地的**实践总结**、技术沙龙与大咖面对面的活动沉淀，以及工程师成长背后的**技术文化思考**，当然还有福利！！！

宠粉时刻

✅ 小红书平台扫码（第1个码）关注【美团技术团队】（赞、评论随心意）
✅ 保存你的“关注截图”→扫二维码（第2个码）抽奖→上传截图→等待开奖！
🎁 吨吨桶、野营系列水壶斜挎小包、野营系列小马扎、长款桌垫、团团捏捏本等30份礼品，**12月5日中午12点开奖**！

---------- END ----------

推荐阅读

- 复旦 NLP&美团 LongCat 联合提出长程推理能力评测与增强新框架
- LongCat 发布 AMO-Bench：重新定义 LLM 数学上限
- 美团 LongCat Interaction 团队发布大模型交互系统技术报告 WOWService

美团技术团队
20000+工程师，如何支撑中国领先的生活服务电子商务平台？数亿消费者、数百万...
551篇原创内容

公众号